Solución S04: protección CSRF en la API

Fecha: 9 de noviembre de 2026. Rama: beta. Base: 7af8274. Estado: corrección validada localmente; despliegue en producción pendiente.

Causa y alcance

Una página en otro origen del mismo sitio podía emitir un POST simple con credentials:include. El navegador adjuntaba la cookie del repartidor y la API ejecutaba la acción. SameSite=Lax no separa esos orígenes. CORS impedía leer la respuesta, pero no impedía el cambio. La condición de explotación era controlar un origen hermano, por ejemplo otro subdominio del mismo dominio registrable; no se demostró acceso desde cualquier sitio.

Corrección

Core/CsrfProtection.cs valida las escrituras de /api antes de ejecutar el endpoint. Program.cs registra la protección antiforgery estándar de ASP.NET Core y coloca el middleware después de autenticar, autorizar y limitar solicitudes. La ruta seleccionada se conoce mediante UseRouting para identificar la única excepción de notificación externa.

GET /api/auth/csrf devuelve requestToken en JSON y crea una cookie host-only diversify.csrf, HttpOnly y SameSite=Strict. En producción requiere Secure. El token se almacena solo en memoria del módulo de JavaScript, se transmite como X-CSRF-TOKEN y se valida contra su cookie y la identidad autenticada. No se envía en la URL ni se guarda en localStorage. La respuesta de tokens no se almacena en caché y se limita por IP.

Las escrituras requieren el token incluso si no tienen cuerpo. Se incluyen login, registro, logout, perfil, solicitud y restablecimiento de contraseña, pedidos, direcciones y administración. La ausencia de token, un token falso, una cookie antiforgery ausente o una identidad diferente producen HTTP 403 con X-CSRF-Error: 1, sin ejecutar el endpoint. Se mantienen los 401/403 de autenticación/autorización cuando esas comprobaciones fallan antes.

Como defensa adicional se rechazan Origin distintos al origen efectivo exacto de la app y Sec-Fetch-Site con same-site o cross-site. Se normalizan scheme/host/puerto, sin permitir todos los subdominios. Los clientes HTTP sin esas cabeceras siguen necesitando token y cookie válidos. La emisión del token también rechaza otros orígenes. La terminación HTTPS debe conservar el esquema/host mediante los proxies de confianza ya configurados.

El frontend wwwroot/js/api.js obtiene un único token inicial compartido entre escrituras paralelas. Lo descarta tras cambios de autenticación, 401, /auth/me anónimo o rechazo CSRF. No reintenta automáticamente una escritura: se muestra el error y el siguiente intento explícito obtiene un token válido. Esto evita duplicar acciones.

PixelPay

La única excepción POST se marca por metadata en /api/pixelpay/aviso; no es una excepción para todo un prefijo de rutas. El aviso servidor a servidor conserva su validación de payment_hash, orden, monto y moneda. Los retornos GET firmados de PixelPay conservan su protocolo. No dependen de la cookie de sesión para aprobar una cuenta. Una petición a esa ruta no puede modificar pagos por el solo hecho de estar exenta de CSRF: sigue necesitando la autenticidad exigida por el proveedor. Las escrituras de inscripción y administración sí requieren token CSRF.

Prueba adversarial y regresiones

Nueva suite tests/seguridad-csrf.cjs: 58 comprobaciones correctas.

- POST sin cuerpo ni token, token falso, ausencia de cookie antiforgery y token de otra identidad: rechazados.

- Origin null o distinto, y Fetch Metadata same-site/cross-site: rechazados aun presentando un token válido.

- Login y logout falsificados, PUT de perfil y DELETE de dirección: rechazados; la dirección permanece.

- La emisión de tokens desde otro Origin devuelve 403.

- El aviso de PixelPay sin orden conocida sigue devolviendo su 404 de negocio, sin pasar por CSRF.

- Ataque real en Chromium desde otro puerto del mismo sitio: CDP confirma que salió la cookie de la víctima. Tras el POST, el master verifica que el pedido completo —asignación, fecha y eventos— permanece idéntico.

- El módulo legítimo toma y libera el mismo pedido. Un POST sin cuerpo es válido con token y Origin correcto.

- Logout y nuevo login desde el mismo módulo funcionan sin recargar la página.

- Un 403 CSRF simulado produce un único intento de escritura; el siguiente intento legítimo renueva el token.

El protocolo de Chrome se usó para comprobar los encabezados realmente enviados, sin imprimir valores de cookies ni tokens. La comprobación de que el pedido no cambió es independiente del bloqueo de lectura de CORS.

Regresiones aprobadas con la protección activa:

- 580 comprobaciones de permisos de repartidor, asignación atómica y cupo de pedidos.

- 71 comprobaciones de revocación de sesiones, reinicios y actualización de base existente.

- Registro, pago simulado, compra, preparación, toma y entrega con PIN en Chromium.

- Recuperación de contraseña y cierre de las sesiones anteriores.

- PixelPay simulado: aprobación, cancelación, rechazo y reintento, aviso sin navegador, monto diferente y firmas falsas.

- Compilación -warnaserror, JavaScript y ShellCheck.

Los clientes HTTP de tests usan tests/api-csrf.cjs para obtener tokens reales. No se desactiva antiforgery en pruebas ni se eluden las validaciones del servidor. Las pruebas negativas envían las solicitudes falsas sin ese helper.

Ejecución y despliegue

```
dotnet build src/Diversify.Delivery -warnaserror
npm install --no-save playwright@1.49.1
npx playwright install --with-deps chromium
node tests/seguridad-csrf.cjs
```

Requiere Node 22+, dotnet y Playwright. La suite usa su propio puerto, proceso y base demo bajo test-results/csrf-*; detiene únicamente sus procesos. El límite de login se eleva solo en esa instancia de pruebas. La suite está en CI.

Backend y frontend deben desplegarse juntos en todas las instancias. Los clientes HTTP externos que escriban con sesión deben obtener el token y conservar su cookie antes de escribir. No hay cambios de esquema, paquetes del backend ni secretos nuevos. Las llaves de Data Protection deben conservarse y compartirse según el despliegue.

Las pruebas fueron locales y no validan la configuración pública efectiva de proxies/TLS ni pagos reales. Antiforgery no evita XSS ni el abuso de credenciales válidas por un cliente no navegador. El inventario actualizado está en docs/BUGS-SEGURIDAD.md: S01–S05 corregidos en beta; S06, S07 y S08 pendientes.